



Proyecto #1
Diagrama de Paquetes – Explicación del SO

David Calvo García

Instituto Tecnológico de Costa Rica

IC6600 Principios de Sistemas Operativos

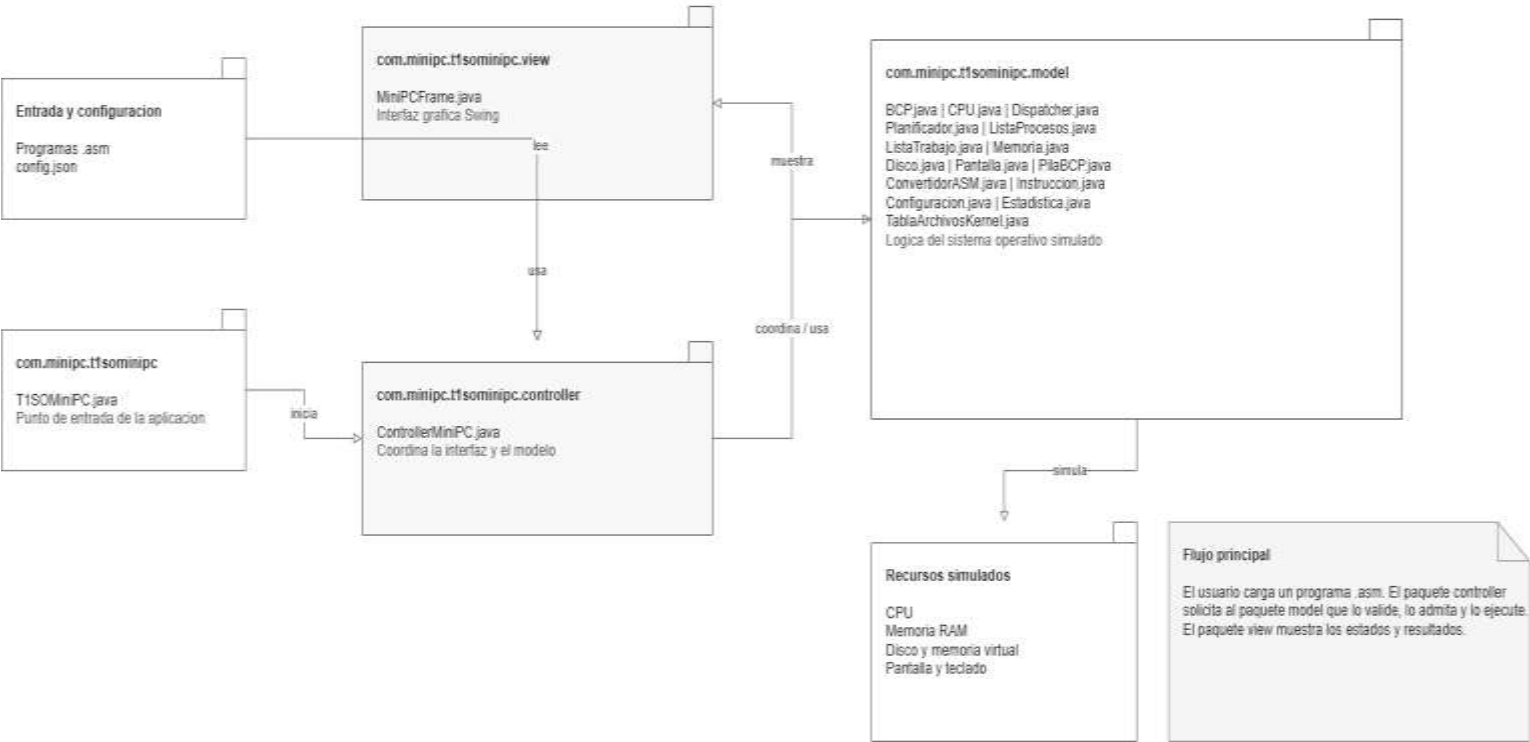
Prof. Cristian Campos Agüero

II Semestre

7 de octubre del 2026

Mini PC - Diagrama de paquetes

com.minipc.tfsominipc



1. Propósito de este documento

Este documento acompaña al diagrama de paquetes anterior y explica, en términos generales, cómo está organizado el código del proyecto y qué decisiones de diseño se tomaron para representar los conceptos de un Sistema Operativo (gestión de procesos, memoria, disco y planificación) dentro de un simulador hecho en Java. No se trata de un manual técnico línea por línea, sino de una guía para entender el porqué detrás de la estructura del proyecto.

2. Qué hace cada paquete

com.minipc.t1sominipc

Es el punto de entrada de la aplicación. Contiene únicamente la clase `T1SOMiniPC.java`, cuyo único trabajo es crear la ventana principal y arrancar el controlador. No tiene ninguna lógica propia del sistema operativo simulado, es solo el "botón de encendido" de la aplicación.

view

Contiene la interfaz gráfica (`MiniPCFrame.java`), construida con Swing. Esta capa solo se encarga de mostrar información y de capturar lo que el usuario hace (cargar un archivo, dar clic en "Paso a paso", escribir un valor de teclado). No decide nada por sí sola ni contiene lógica de ejecución, simplemente refleja lo que el controlador le indica que debe mostrar.

controller

Es el intermediario entre la interfaz y la lógica del sistema operativo simulado (`ControllerMiniPC.java`). Cuando el usuario carga un archivo o presiona un botón, es el controlador quien recibe ese evento, se lo comunica a las clases del paquete `model` para que hagan el trabajo real, y después actualiza la vista con el resultado. Siguiendo el patrón Modelo-Vista-Controlador, esta capa coordina sin contener la lógica de negocio en sí misma.

model

Aquí vive toda la simulación del sistema operativo. Agrupa, entre otras, las siguientes clases:

- CPU: ejecuta las instrucciones, una por una, siguiendo el ciclo fetch-decode-execute.

- Instruccion / ConvertidorASM: representan y validan cada línea del archivo ensamblador antes de que el sistema las acepte.
- Memoria: simula la RAM, dividida en un segmento de kernel y uno de usuario.
- BCP: el Bloque de Control de Proceso de cada programa en ejecución.
- PilaBCP: la pila de 5 posiciones que usa cada proceso para PUSH, POP y PARAM.
- Disco: simula el almacenamiento secundario, con su índice de archivos y su zona de memoria virtual.
- ListaTrabajo / ListaProcesos: las dos colas de procesos que explica la sección 4.
- Planificador / Dispatcher: deciden el orden de ejecución y ejecutan la admisión y el cambio de contexto.
- Pantalla / TablaArchivosKernel / Configuración / Estadística: apoyan la simulación de entrada y salida, la configuración externa y el reporte final.

Entrada y configuración / Recursos simulados

No son paquetes de código en sí, sino los dos extremos del flujo, por un lado, lo que el usuario entrega al sistema (archivos .asm y un archivo de configuración externo con el tamaño de la RAM y del disco, que por requerimiento del proyecto no puede quedar escrito directamente en el código), por el otro, los recursos de hardware que el paquete modela, la CPU, la memoria RAM, el disco con su memoria virtual, y los dispositivos de pantalla y teclado.

3. Flujo principal

El recorrido típico de una ejecución es el siguiente, el usuario carga uno o varios archivos .asm desde la interfaz, el controlador se los entrega al ConvertidorASM, que valida la sintaxis de cada línea y rechaza el archivo completo si encuentra errores. Si el archivo es válido, se guarda en el Disco y se registra en la Lista de Trabajo. A partir de ahí, el Despachador intenta admitirlo a la Lista de Procesos siguiendo los dos pasos que se explican en la sección 4. Una vez admitido y con la CPU libre, el Planificador (FCFS) decide a quién le toca ejecutar, y la interfaz va mostrando en tiempo real los registros, el estado del proceso, el contenido de la memoria y la salida en pantalla.

4. Decisiones de diseño

El BCP vive dentro de la memoria, no como objeto aparte

En vez de que cada BCP sea un objeto Java con sus propios atributos sueltos, cada uno de sus campos (PID, estado, registros, base, tamaño, prioridad, enlace al siguiente BCP, información de la pila, etc.) se escribe como una posición real dentro del arreglo que representa la memoria del kernel. Esto se acerca más a cómo funciona un sistema operativo real, donde la tabla de procesos vive físicamente en RAM, y permite que la interfaz muestre, celda por celda, exactamente dónde y cómo está guardado cada proceso.

Memoria dividida en kernel y usuario

La memoria principal es un único arreglo, pero se reserva automáticamente un porcentaje para el kernel (donde viven los BCP y la lista de trabajo) y el resto queda disponible para que los programas de usuario se carguen y ejecuten. El tamaño total es configurable desde fuera del código, tal como lo pide el proyecto.

Disco con tres secciones

El almacenamiento secundario se divide, dentro del mismo espacio de direcciones, en tres zonas: un índice de archivos al inicio (con el nombre y la dirección de cada programa guardado), una zona donde viven los programas en sí, y una última zona reservada que cumple el papel de memoria virtual.

Admisión de procesos en dos pasos

Antes de que un programa pase de la Lista de Trabajo a la Lista de Procesos, el Despachador verifica dos condiciones en orden: primero, si hay espacio disponible en el kernel para crear su BCP, si no lo hay, el trabajo se queda esperando su turno. Si el BCP sí se puede crear, se revisa después si el programa cabe en la memoria de usuario (RAM). Cuando no cabe, el programa se copia a la zona de memoria virtual del disco y el proceso queda en estado "En espera", hasta que se libera suficiente RAM y se realiza el intercambio (swap-in) de vuelta a memoria principal.

Dos listas, no una sola

La Lista de Trabajo contiene todos los programas que el usuario ha cargado, tengan o no BCP asignado todavía. La Lista de Procesos contiene únicamente a los que ya tienen BCP, con un límite real de hasta 5 procesos simultáneos. Separar ambas listas evita gastar espacio de kernel en programas que todavía ni siquiera han sido evaluados para su admisión.

Planificación FCFS sin apropiación

El algoritmo implementado es FCFS (el primero en llegar es el primero en ejecutarse). Un proceso que ya está en ejecución no es interrumpido por otro proceso, ni siquiera cuando se pausa esperando un valor de teclado (INT 09H), simplemente espera a que el usuario ingrese el dato y continúa donde se quedó. El cambio hacia otro proceso solo ocurre cuando el que está corriendo termina por completo.

Planificador y Despachador, responsabilidades separadas

El Planificador decide el orden de ejecución según el algoritmo vigente (hoy FCFS), el Despachador es quien efectivamente mueve los datos: crea el BCP, hace el intercambio con memoria virtual cuando corresponde, y asigna los registros de la CPU al proceso elegido. Mantenerlos separados permite, en un futuro, cambiar el algoritmo de planificación sin tener que tocar el mecanismo de administración de memoria y contexto.

5. Protección y seguridad

Cada proceso solo puede leer y escribir dentro del rango de memoria que se le asignó al momento de su admisión (su dirección base y su tamaño). Si una instrucción de salto (JMP, JE, JNE) intenta dirigirse fuera de ese rango, la CPU rechaza el salto y lo reporta como un error controlado, sin detener el resto del sistema. De la misma forma, si un proceso avanza su contador de programa más allá de su propio espacio sin haber terminado con la instrucción INT 20H, se finaliza de forma forzosa para no invadir la memoria de otro proceso. La pila de cada proceso tiene un tamaño fijo de 5 posiciones y controla explícitamente el desbordamiento (PUSH) y el vaciado (POP), devolviendo un aviso en lugar de permitir un acceso inválido.